**DAY 21 – FILE PERMISSION ANALYSIS**

**Cyber Security Internship – Veda Technology**

1. Objective

To analyze Linux file permissions, understand access control for owners, groups, and others, and identify overly permissive configurations that may expose files and directories to unauthorized access.

2. Introduction

Linux is a multi-user operating system that uses file permissions to protect system resources and sensitive information.

Every file and directory in Linux has associated permissions that determine which users can read, modify, or execute it.

Linux permissions are divided into three categories:

- Owner: The user who owns the file or directory.
- Group: Users belonging to the file's assigned group.
- Others: Users who are neither the owner nor members of the assigned group.

Improper permission settings can result in unauthorized data access, modification, or execution of malicious programs.

File permission analysis helps identify these security weaknesses and implement appropriate access restrictions.

3. Understanding Linux File Permissions

Linux uses three basic permission types.

| Symbol | Permission | Numeric Value | Description |
|---|---|---|---|
| r | Read | 4 | Allows reading file contents |
| w | Write | 2 | Allows modifying file contents |
| x | Execute | 1 | Allows executing files |
| - | No access | 0 | Permission not granted |

Common Linux Permission Configurations

| Permission | Symbolic Format | Description |
|---|---|---|
| 777 | rwxrwxrwx | Everyone has full access |
| 755 | rwxr-xr-x | Owner has full access; others can read and execute |

| | | |
|---|---|---|
| 700 | rwx------ | Only owner has full access |
| 666 | rw-rw-rw- | Everyone can read and modify |
| 644 | rw-r--r-- | Owner can modify; others can read |
| 600 | rw------- | Only owner can read and modify |

Security Observation: Permissions such as 777 and 666 are generally unsafe for sensitive files because they allow unnecessary access.

4. Practical Implementation

The following commands demonstrate Linux file permission analysis in a practice environment.

Step 1: Create a Practice Directory

Commands:

mkdir Day21_PermissionAnalysis

cd Day21_PermissionAnalysis

touch public.txt private.txt script.sh

Description:

A directory named Day21_PermissionAnalysis is created with three sample files for testing different permission configurations.

Step 2: View Existing File Permissions

Command:

ls -l

Description:

The ls -l command displays file permissions, ownership, group information, size, and modification date.

This helps identify files that may have unnecessary access permissions.

Step 3: Assign Different Permission Levels

Commands:

chmod 777 public.txt

chmod 600 private.txt

chmod 755 script.sh

ls -l

Description:

Different permissions are assigned to compare secure and overly permissive configurations.

| File | Permission | Security Analysis |
|---|---|---|
| public.txt | 777 | High risk: all users can modify the file |
| private.txt | 600 | Restricted to owner |
| script.sh | 755 | Executable by all users; appropriate only when intended |

Step 4: Identify Overly Permissive Files

Command:

find . -type f -perm -0002

Description:

This command searches the current directory and its subdirectories for regular files that have write permission enabled for others.

Expected output:

./public.txt

Observation: The file public.txt is identified as potentially insecure because users outside the owner and assigned group can modify its contents.

Step 5: Correct Insecure Permissions

Commands:

chmod 644 public.txt

ls -l public.txt

Description:

The permission of public.txt is changed from 777 to 644.

The owner retains read and write access, while the group and others receive read-only access.

Expected output (permission portion):

-rw-r--r--

This reduces the risk of unauthorized file modification.

Step 6: Verify File Permissions

Command:

ls -l

Expected permission results:

| File | Final Permission | Access Level |
|---|---|---|
| public.txt | 644 | Owner write; others read |
| private.txt | 600 | Owner only |
| script.sh | 755 | Owner write; everyone read and execute |

The corrected permissions demonstrate how access can be restricted according to file requirements.

5. File Permission Security Analysis

| Configuration | Security Risk | Recommendation |
|---|---|---|
| 777 | Any user can modify or execute | Avoid for regular files |
| 666 | Any user can modify contents | Restrict write access |
| 755 | Everyone can read and execute | Use only when public execution is intended |
| 644 | Contents readable by all users | Use for non-sensitive documents |
| 600 | Access restricted to owner | Suitable for confidential files |
| 700 | Only owner has full access | Suitable for private scripts and directories |

6. Security Risks of Excessive Permissions

1. Unauthorized Modification: Other users may alter or delete important file contents.
2. Data Exposure: Sensitive information may become accessible to unintended users.
3. Malicious Script Modification: Writable scripts can be altered to execute harmful commands.
4. Privilege Escalation: Insecure permissions on privileged files or scripts can contribute to privilege escalation.
5. System Misconfiguration: Excessive access can weaken the operating system's overall security.

7. Best Practices for File Permission Management

- Follow the principle of least privilege.
- Avoid assigning 777 permissions unnecessarily.
- Use 600 for sensitive files requiring owner-only access.
- Use 644 for non-sensitive files intended to be publicly readable.
- Regularly inspect file permissions using ls -l.

- Identify world-writable files using the find command.
- Assign execute permissions only when required.
- Review file ownership and group permissions periodically.

8. Results and Observations

The illustrative permission analysis demonstrates how Linux access controls can be examined and corrected.

| Test Activity | Expected Result |
|---|---|
| File creation | Three sample files created |
| Permission inspection | Existing access rights displayed |
| Insecure permission assignment | public.txt configured with 777 |
| Permission scanning | World-writable public.txt identified |
| Permission correction | public.txt changed to 644 |
| Final verification | Restricted permissions displayed |

The demonstration highlights how inappropriate permissions can create security risks and how Linux commands can be used to correct them.

9. Conclusion

The File Permission Analysis task provided an understanding of Linux file permissions and their importance in operating system security.

Commands such as ls -l, chmod, and find were used to demonstrate how to inspect, modify, and analyze file permissions.

The task highlighted that overly permissive configurations such as 777 can expose files to unauthorized modifications, while restrictive permissions such as 600 and 644 provide better access control when appropriately selected.

Overall, this exercise reinforced the importance of regular permission auditing, secure configuration, and the principle of least privilege in maintaining Linux system security.

